

ECON 609: Optimization

Minsu Chang

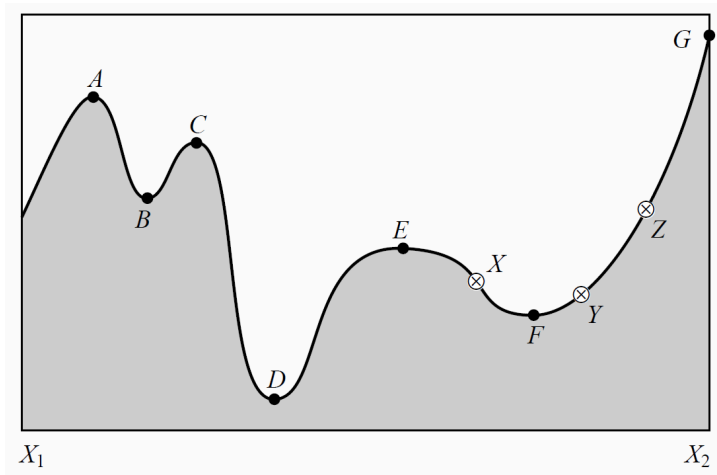
Georgetown University

Lecture # 1

Optimization

- ▶ Optimization of functions is at the core of most economic models: fundamental behavioral assumption of agents.
- ▶ Also, key for most methods is classical econometrics.
- ▶ Nowadays: machine learning → large optimization problems that require efficient computation.
- ▶ We rarely have closed-form solutions. Numerical optimization!

The Challenge



Overview

- (1) Methods using derivatives
- (2) Methods without using derivatives
- (3) Mixed methods
- (4) Simulation methods

Newton-Raphson Method

- ▶ Most common root-finding method, commonly used in economics with first-order conditions.
- ▶ Works with univariate and multivariate optimization problems, but requires twice-differentiability of function.
- ▶ Optimize $f(x)$ by using successive quadratic approximations to it.

Idea: Univariate case

- ▶ Optimize $f(x)$ where the first-order condition $f'(x) = 0$.
- ▶ Given an initial guess x_0 , compute the first-order Taylor approximation of $f'(x)$ around x_0 :

$$f'(x) \approx f'(x_0) + f''(x_0)(x - x_0)$$

- ▶ The idea is to update with a tangent line.

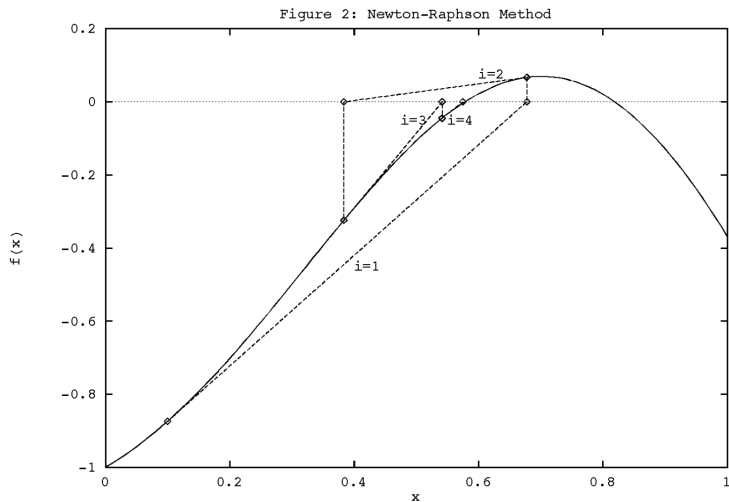
$$f'(x_0) + f''(x_0)(x^* - x_0) = 0$$

which gives $x^* = x_0 - \frac{f'(x_0)}{f''(x_0)}$.

- ▶ This suggests the iteration

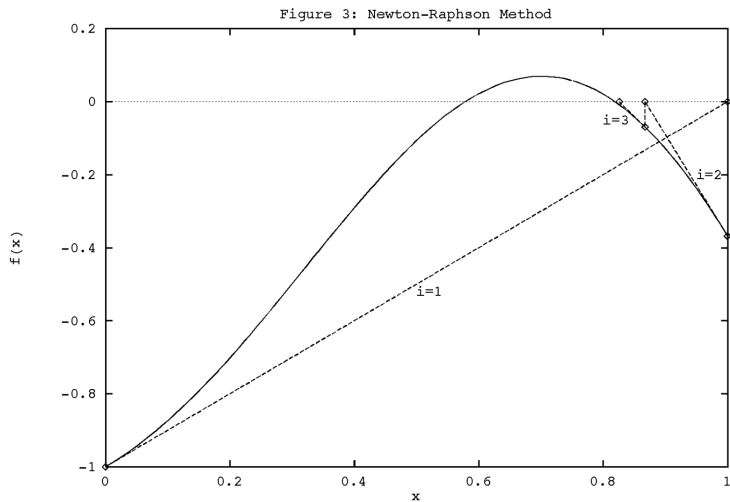
$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

Newton-Raphson Method



Reference: https://web.mit.edu/10.001/Web/Course_Notes/NLAE/node6.html

Newton-Raphson Method



Reference: https://web.mit.edu/10.001/Web/Course_Notes/NLAE/node6.html

Idea: Multivariate case

- ▶ For a N -dimensional vector function $f(x)$, $x \in \mathbb{R}^N$, we can follow the same steps.

- ▶ We get

$$x_{n+1} = x_n - H_n^{-1} \nabla f(x_n)$$

where $\nabla f(\cdot)$ is the gradient and $H(\cdot)$ is the Hessian.

- ▶ Problems:
 - ▶ Numerical evaluation of Hessian: curse of dimensionality
 - ▶ Sensitive with respect to initial guess.

BFGS to approximate Hessian

- Broyden-Fletcher-Goldfarb-Shanno (BFGS) developed an efficient algorithm to approximate the Hessian:

$$H_{n+1} = H_n + \frac{yy^T}{y^T s} - \frac{H_n s s^T H_n^T}{s^T H_n s}$$

$$s = x_{n+1} - x_n$$

$$y = \nabla f(x_{n+1}) - \nabla f(x_n)$$

- Furthermore,

$$H_{n+1}^{-1} = \left(I - \frac{sy^T}{y^T s} \right) H_n^{-1} \left(I - \frac{ys^T}{y^T s} \right) + \frac{ss^T}{y^T s}$$

- This is computationally efficient since taking inverse of matrices is very slow.

Grid Search

- ▶ We define a grid $[x_1, x_2, \dots, x_N]$ with N points.
- ▶ We check the function $f(\cdot)$ at each point of the grid.
- ▶ We keep the lowest (highest) value.
- ▶ Suffer from curse of dimensionality and it may fail if the grid too coarse.
- ▶ But under certain condition it can be quite useful:
 - ▶ Discrete choices
 - ▶ Monotonicity that we can exploit
 - ▶ Easy to parallelize

Golden Section Search

- ▶ Find minimum x of unimodal continuous $f : X \rightarrow R$ in an interval $[a, c]$.
- ▶ Assume $\exists x \in (a, c)$ and $f(x) < \min[f(a), f(c)]$.
- ▶ Idea:
 - (1) Select triplet (a, b, c)
 - (2) Update triplet to (a', b', c') with narrower value range that includes minimum.
 - (3) Stop when value range is narrow enough.
- ▶ How do we pick (a, b, c) ? How do we update triplet (a, b, c) ?

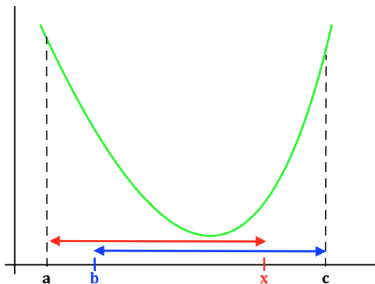
Algorithm

- (1) Set $b = a + \frac{3-\sqrt{5}}{2}(c - a)$.
- (2) Set $x = a + \frac{\sqrt{5}-1}{2}(c - a)$.
- (3) If $|x - b| < tol$, then exit the algorithm with return $\frac{x+b}{2}$. If not, go to (4).
- (4) If $f(b) < f(x)$, update triplet to (a, b, x) and go to (1). Otherwise, update triplet to (b, x, c) and go to (1).

Graphical Illustration

- ▶ Set triplet (a, b, c) .
- ▶ Choose x such that red and blue lines are equal.
- ▶ Golden section: relative size of both lines is a particular number.

$$\frac{x - a}{c - a} = \frac{c - b}{c - a} = \frac{\sqrt{5} - 1}{2} \approx 0.618$$



- ▶ No analogous procedure in multidimensional space.

Direction Set Methods

- ▶ Suppose our function $f(x)$, $x \in \mathbb{R}^N$ is roughly approximated as a quadratic form:

$$f(x) \approx \frac{1}{2}x^T A x - b^T x + c$$

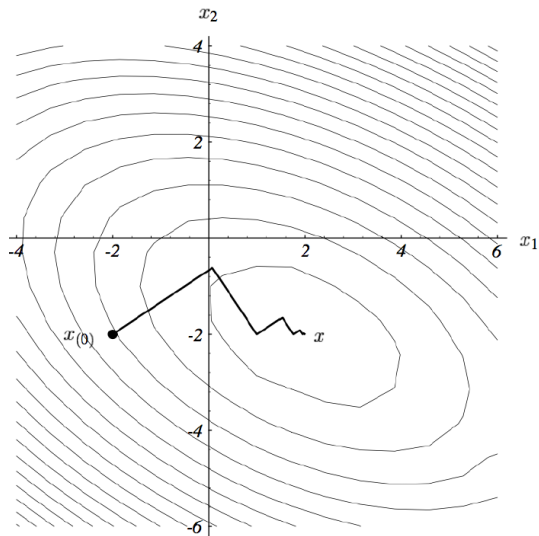
A is a known, square, symmetric, positive-definite matrix.

- ▶ Then, $f(x)$ is minimized by the solution to $Ax = b$.
- ▶ Suppose we can calculate $f(P)$ and $\nabla f(P)$ for a given N -dimensional point P .
- ▶ How can we use this additional information?

Steepest Descent Method

- ▶ A tempting (but not very good) possibility: steepest descent method
- ▶ Start at a point P_0 . As many times as needed, move from point P_i to the point P_{i+1} by minimizing along the line from P_i in the direction of the local downhill gradient $-\nabla f(P_i)$.
- ▶ Performs many small steps: not very efficient!

Steepest Descent Method



Conjugate Gradient Method

- ▶ A better way.
- ▶ In $\mathbb{R}^N$, take N steps each of which attains the minimum in each direction, without undoing previous steps' progress.
- ▶ In other words, proceed in a direction that is somehow constructed to be conjugate to the old gradient, and to all previous directions traversed.

Algorithm

(1) Let $d_0 = r_0 = -f'(x_0)$.

(2) For $i = 0, 1, \dots, N - 1$, do:

▶ $\alpha_i = \frac{d_i^T r_i}{d_i^T A d_i}.$

▶ $x_{i+1} = x_i + \alpha_i d_i.$

▶ $r_{i+1} = -f'(x_{i+1}).$

▶ $\beta_{i+1} = \frac{r_{i+1}^T r_{i+1}}{r_i^T r_i}$

▶ $d_{i+1} = r_{i+1} + \beta_{i+1} d_i.$

(3) Return x_N .

Random Walk Metropolis-Hastings

- ▶ We explore a function $f(\cdot)$ by randomly drawing from it.
- ▶ Algorithm:

(1) Given a state of the chain x_{n-1} , we generate a proposal:

$$x^* = x_{n-1} + \lambda \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

(2) We compute $\alpha = \min \left\{ 1, \frac{f(x^*)}{f(x_{n-1})} \right\}$.

(3) We set

$$x_n = x^* \quad \text{w.p. } \alpha$$

$$x_n = x_{n-1} \quad \text{w.p. } 1 - \alpha$$

(4) Keep x_n which yields the highest $f(\cdot)$.

- ▶ Equivalent to simulated annealing: iteration-varying λ .

Introduction to Bayesian Inference

- ▶ Ingredients of Bayesian Analysis:

- ▶ Likelihood function $p(Y|\phi)$

- ▶ Prior density $p(\phi)$

- ▶ Marginal data density $P(Y) = \int p(Y|\phi)p(\phi)d\phi$

- ▶ Bayes theorem:

$$p(\phi|Y) = \frac{p(Y|\phi)p(\phi)}{p(Y)}$$

- ▶ Posterior $p(\phi|Y)$: probability of the parameter given the data.

Linear Regression/ AR Models

- ▶ Consider AR(1) model:

$$y_t = y_{t-1}\phi + u_t, \quad u_t \sim_{i.i.d} N(0, 1)$$

- ▶ Let $x_t = y_{t-1}$. Write as

$$y_t = x_t\phi + u_t, \quad u_t \sim_{i.i.d} N(0, 1)$$

or $Y = X\phi + U$.

- ▶ Likelihood function:

$$p(Y|\phi) = (2\pi)^{-\frac{T}{2}} \exp \left\{ -\frac{1}{2} (Y - X\phi)' (Y - X\phi) \right\}$$

A Convenient Prior

- Prior:

$$\phi \sim N(0, \tau^2), \quad p(\phi) = (2\pi\tau^2)^{-1/2} \exp\left\{-\frac{1}{2\tau^2} \phi' \phi\right\}$$

- Large τ means diffuse prior.
- Small τ means tight prior.

Deriving the Posterior

- Bayes theorem:

$$\begin{aligned} p(\phi|Y) &\propto p(Y|\phi)p(\phi) \\ &\propto \exp \left\{ -\frac{1}{2}[(Y - X\phi)'(Y - X\phi) + \tau^{-2}\phi'\phi] \right\} \end{aligned}$$

- Guess: If $\phi|Y \sim N(\bar{\phi}_T, \bar{V}_T)$, then

$$p(\theta|Y) \propto \exp \left\{ -\frac{1}{2}(\phi - \bar{\phi}_T)' \bar{V}_T^{-1} (\phi - \bar{\phi}_T) \right\}$$

- Rewrite exponential term

$$\begin{aligned} Y'Y - \phi'X'Y - Y'X\phi + \phi'X'X\phi + \tau^{-2}\phi'\phi \\ = Y'Y - \phi'X'Y - Y'X\phi + \phi'(X'X + \tau^{-2}\mathcal{I})\phi \end{aligned}$$

Deriving the Posterior

- ▶ Exponential term is a quadratic function of ϕ .
- ▶ Deduce: posterior distribution of ϕ is a multivariate normal distribution $\phi|Y \sim N(\bar{\phi}_T, \bar{V}_T)$ with

$$\bar{\phi}_T = (X'X + \tau^{-2}\mathcal{I})^{-1}X'Y$$

$$\bar{V}_T = (X'X + \tau^{-2}\mathcal{I})^{-1}$$

- ▶ In this example, directly sampling from the posterior is possible. In general, it is not feasible.
- ▶ With more complex posterior distributions, we can generate draws $\{\phi^i\}_{i=1}^N$ using random walk Metropolis-Hastings.